

LangSmith 全链路观测：从 Agent 调试到 RAG 量化评估 已付费

原创 神说要有光 神光的幸福生活 2026年5月16日 21:53 山东

我们学了基于 LangChain 、LangGraph 开发 Agent，但总有一种强烈的“盲盒感”。

现在可以看到终端不断流出的 Token，但是：

它调用了哪个工具？每一步耗时多少？消耗了多少 token？

而且，当你试图复现那个偶尔出现的 Bug 时，它又消失了。

软件工程界有一句名言：“如果你无法度量它，你就无法管理它。”

所以我们要给 Agent 加上全生命周期的可观测性。

这就是 LangSmith 。

如果说大模型是引擎，那么 LangSmith 就是那个不可或缺的仪表盘。

这节我们就来学一下 LangSmith

首先打开 langsmith 的网站：

<https://smith.langchain.com/>

点击左下角 setting 创建 api key

然后我们在项目 .env 里加几个环境变量：

```
# 用于身份验证，实现链路上报
LANGCHAIN_API_KEY=你的key
# 指定LangSmith中的项目，追踪结果会归类到该项目下
LANGCHAIN_PROJECT=项目名
# 开启LangSmith追踪功能
LANGCHAIN_TRACING_V2=true
```

LANGCHAIN_API_KEY 就是你刚才创建的 api key，可以标识哪个用户

LANGCHAIN_PROJECT 是用户的哪个项目

LANGCHAIN_TRACING_V2 是开启追踪

只要加上这几个环境变量跑，不需要做什么，就会自动上报 trace 数据：

神光的编程秘籍

这个是 trigger-error.mjs

```
import "dotenv/config";
import { Annotation, END, START, StateGraph } from "@langchain/langgraph";

/**
```

```

* 故意在图节点里抛错, 用于验证 LangSmith / 本地日志是否能看到失败 run。
* 运行: node src/trigger-error.mjs
*
* 若需要「未捕获的 Promise rejection」观察进程行为, 可删掉下方 try/catch,
* 仅保留 await graph.invoke(...)。
*/

const StateAnnotation = Annotation.Root({
  text: Annotation({
    reducer: (_prev, next) => next,
    default: () => "",
  }),
});

const stepOk = (state) => ({ text: `${state.text}[ok]` });

const stepThrow = () => {
  throw new Error("DemoError: 节点内故意抛错 (trigger-error.mjs) ");
};

const graph = new StateGraph(StateAnnotation)
  .addNode("step_ok", stepOk)
  .addNode("step_throw", stepThrow)
  .addEdge(START, "step_ok")
  .addEdge("step_ok", "step_throw")
  .addEdge("step_throw", END)
  .compile();

try {
  await graph.invoke({ text: "start" });
  console.log("不应执行到这里");
} catch (err) {
  console.error("已捕获:", err?.message ?? err);
  process.exitCode = 1;
}

```

我们收集了几个 graph 的运行的 trace 数据, 看到了整体统计的 monitor 数据。

我们 Agent 的运行情况就一目了然了, 非常方便就接入了全链路的观测。

除了日常监控观测之外, 我们还需要对业务效果做标准化评估。

这时候就可以使用 LangSmith 的 Dataset 功能, 也就是测试样本, 统一存放用户提问和标准答案。

搭建好数据集后, 再通过 Evaluation 设定打分规则, 批量完成自动化评测, 精准衡量回答质

量，用量化数据来优化 Agent 和 RAG 相关业务逻辑。

我们写个 rag 的 agent 案例：

```
mkdir langsmith-test
cd langsmith-test
npm init -y
```

安装依赖：

```
pnpm install @langchain/core @langchain/langgraph @langchain/community @langchain/openai
```

创建 .env

```
OPENAI_API_KEY=sk-xxx
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
MODEL_NAME=qwen-plus

# Milvus
MILVUS_URI=http://localhost:19530
MILVUS_COLLECTION=rag_docs

# Embedding
EMBEDDING_MODEL=text-embedding-v3

# 用于身份验证，实现链路上报
LANGCHAIN_API_KEY=xxx
```

```
# 指定LangSmith中的项目，追踪结果会归类到该项目下
LANGCHAIN_PROJECT=langsmith-test

# 开启LangSmith追踪功能
LANGCHAIN_TRACING_V2=true
```

然后我们先用 docker compose 把 Milvus 跑起来：

docker-compose.yml

```
services:

# Milvus
etcd:
  container_name:etcd-container
  image:quay.io/coreos/etcd:v3.5.18
  environment:
    -ETCD_AUTO_COMPACTION_MODE=revision
    -ETCD_AUTO_COMPACTION_RETENTION=1000
    -ETCD_QUOTA_BACKEND_BYTES=4294967296
    -ETCD_SNAPSHOT_COUNT=50000
  volumes:
    -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/etcd:/etcd
  command:etcd-advertise-client-urls=http://etcd:2379-listen-client-urlshttp://0.0.0.0:
  healthcheck:
    test:["CMD","etcdctl","endpoint","health"]
    interval:30s
    timeout:20s
    retries:3

minio:
  container_name:minio-container
  image:minio/minio:RELEASE.2024-05-28T17-19-04Z
  environment:
    MINIO_ACCESS_KEY:minioadmin
    MINIO_SECRET_KEY:minioadmin
  ports:
    -"9001:9001"
    -"9000:9000"
  volumes:
    -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/minio:/minio_data
  command:minioserver/minio_data--console-address":9001"
  healthcheck:
    test:["CMD","curl","-f","http://localhost:9000/minio/health/live"]
    interval:30s
    timeout:20s
```

```

    retries:3

standalone:
  container_name:standalone
  image:milvusdb/milvus:v2.5.25
  command:["milvus","run","standalone"]
  security_opt:
    -seccomp:unconfined
  environment:
    MINIO_REGION:us-east-1
    ETCD_ENDPOINTS:etcd:2379
    MINIO_ADDRESS:minio:9000
  volumes:
    -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus
  healthcheck:
    test:["CMD","curl","-f","http://localhost:9091/healthz"]
    interval:30s
    start_period:90s
    timeout:20s
    retries:3
  ports:
    -"19530:19530"
    -"9091:9091"
  depends_on:
    -"etcd"
    -"minio"

networks:
default:
  name:common-network

```

创建插入数据的代码:

src/milvus_insert.mjs

```

import "dotenv/config";
import { existsSync, readFileSync, readdirSync } from"fs";
import { join } from"path";
import { MilvusClient, DataType, IndexType, MetricType } from"@zilliz/milvus2-sdk-node";
import { RecursiveCharacterTextSplitter } from"@langchain/textsplitters";
import { OpenAIEmbeddings } from"@langchain/openai";

const COLLECTION = process.env.MILVUS_COLLECTION ?? "rag_docs";
const MILVUS_ADDRESS =
  process.env.MILVUS_URI?.replace(/^https?:\/\//, "") ?? "localhost:19530";

```

```

const embeddings = new OpenAIEmbeddings({
  apiKey: process.env.OPENAI_API_KEY,
  model: process.env.EMBEDDING_MODEL ?? "text-embedding-v3",
  configuration: { baseUrl: process.env.OPENAI_BASE_URL },
});

const client = new MilvusClient({ address: MILVUS_ADDRESS });

asyncfunction loadChunks(dataDir = "./data") {
  if (!existsSync(dataDir)) {
    thrownewError(`数据目录不存在: ${dataDir}`);
  }
  const files = readdirSync(dataDir).filter((f) =>/\.(txt|md)$/i.test(f));
  if (files.length === 0) {
    thrownewError(`目录内无 .txt/.md 文件: ${dataDir}`);
  }

  const docs = files.map((f) => ({
    pageContent: readFileSync(join(dataDir, f), "utf-8"),
    metadata: { source: f },
  }));

  const splitter = new RecursiveCharacterTextSplitter({
    chunkSize: 500,
    chunkOverlap: 50,
  });
  return splitter.splitDocuments(docs);
}

asyncfunction main() {
  try {
    console.log("Connecting to Milvus...");
    await client.connectPromise;
    console.log("✓ Connected\n");

    const chunks = await loadChunks();

    if ((await client.hasCollection({ collection_name: COLLECTION })).value) {
      await client.dropCollection({ collection_name: COLLECTION });
      console.log(`Dropped collection: ${COLLECTION}\n`);
    }

    console.log("Generating embeddings...");
    const vectors = await embeddings.embedDocuments(
      chunks.map((c) => c.pageContent),
    );
  }
}

```

```

const dim = vectors[0].length;

console.log("Creating collection...");
await client.createCollection({
  collection_name: COLLECTION,
  fields: [
    {
      name: "langchain_primaryid",
      data_type: DataType.Int64,
      is_primary_key: true,
      autoID: true,
    },
    { name: "langchain_vector", data_type: DataType.FloatVector, dim },
    { name: "langchain_text", data_type: DataType.VarChar, max_length: 8000 },
    { name: "source", data_type: DataType.VarChar, max_length: 256 },
  ],
});
console.log("Collection created");

console.log("\nCreating index...");
await client.createIndex({
  collection_name: COLLECTION,
  field_name: "langchain_vector",
  index_type: IndexType.IVF_FLAT,
  metric_type: MetricType.L2,
  params: { nlist: 128 },
});
console.log("Index created");

console.log("\nLoading collection...");
await client.loadCollection({ collection_name: COLLECTION });
console.log("Collection loaded");

console.log("\nInserting...");
const data = chunks.map((chunk, i) => ({
  langchain_text: chunk.pageContent,
  langchain_vector: vectors[i],
  source: chunk.metadata.source,
}));

const result = await client.insert({
  collection_name: COLLECTION,
  data,
});
console.log(`✓ Inserted ${result.insert_cnt} records\n`);
} catch (error) {
  console.error("Error:", error.message);
}

```



```
        process.exit(1);
    }
}

main();
```

跑一下:

神光的编程秘籍

然后来写一下 RAG 检索的 Agent:

src/rag_agent.mjs

```
import "dotenv/config";
import { Annotation, END, START, StateGraph } from "@langchain/langgraph";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { RunnableSequence } from "@langchain/core/runnables";
import { ChatOpenAI, OpenAIEmbeddings } from "@langchain/openai";
import { Milvus } from "@langchain/community/vectorstores/milvus";

const embeddings = new OpenAIEmbeddings({
  apiKey: process.env.OPENAI_API_KEY,
  configuration: { baseURL: process.env.OPENAI_BASE_URL },
  model: process.env.EMBEDDING_MODEL ?? "text-embedding-v3",
});

const llm = new ChatOpenAI({
```

```

apiKey: process.env.OPENAI_API_KEY,
configuration: { baseUrl: process.env.OPENAI_BASE_URL },
model: process.env.MODEL_NAME ?? "qwen-plus",
temperature: 0,
});

const vectorStore = await Milvus.fromExistingCollection(embeddings, {
  collectionName: process.env.MILVUS_COLLECTION ?? "rag-docs",
  url: process.env.MILVUS_URI ?? "http://localhost:19530",
});

const retriever = vectorStore.asRetriever({ k: 4 });

const prompt = ChatPromptTemplate.fromMessages([
  [
    "system",
    "你是客服助手。仅根据下面「上下文」回答；上下文没有的信息请明确说不知道，不要编造。\\n\\n上下文: \\n{co
  ],
  ["human", "{question}"],
]);

const chain = RunnableSequence.from([prompt, llm, new StringOutputParser()]);

const GraphState = Annotation.Root({
  question: Annotation,
  context: Annotation,
  answer: Annotation,
});

asyncfunction retrieve(state) {
  const docs = await retriever.invoke(state.question);
  return { context: docs };
}

asyncfunction generate(state) {
  const contextText = state.context.map((d) => d.pageContent).join("\\n\\n");
  const answer = await chain.invoke({
    context: contextText,
    question: state.question,
  });
  return { answer };
}

const workflow = new StateGraph(GraphState)
  .addNode("retrieve", retrieve)
  .addNode("generate", generate)
  .addEdge(START, "retrieve")

```

```

    .addEdge("retrieve", "generate")
    .addEdge("generate", END);

export const ragApp = workflow.compile();

export async function ask(question) {
  const result = await ragApp.invoke({ question });
  return {
    answer: result.answer,
    context: result.context ?? [],
  };
}

```

这里就是一个检索节点、一个生成节点，用来检索向量数据库，生成回答。

然后加一个 cli.mjs

```

import "dotenv/config";
import { ask } from "./rag_agent.mjs";

const DEFAULT_QUESTIONS = [
  "无理由退货要在几天内？ ",
  "满多少元包邮？ ",
  "金卡会员有什么折扣？ ",
  "电子发票多久能开好？ ",
  "手机保修多久？ ",
  "紧急问题怎么联系客服？ ",
];

const args = process.argv.slice(2);
const questions = args.length > 0 ? [args.join(" ")] : DEFAULT_QUESTIONS;

function printContext(context) {
  if (!context.length) {
    console.log("\n引用片段：（无）");
    return;
  }
  // console.log("\n引用片段：");
  // context.forEach((doc, i) => {
  //   const source = doc.metadata?.source ?? "未知";
  //   const text = doc.pageContent.replace(/\s+/g, " ").trim();
  //   const preview = text.length > 100 ? `${text.slice(0, 100)}...` : text;
  //   console.log(`  [${i + 1}] ${source}`);
  //   console.log(`    ${preview}`);
  // });
}

```

```
for (let i = 0; i < questions.length; i++) {  
  const question = questions[i];  
  console.log(`\n${"=".repeat(50)}`);  
  console.log(`问题 ${i + 1}: ${question}`);  
  
  const { answer, context } = await ask(question);  
  console.log(`\n答: ${answer}`);  
  printContext(context);  
}  
  
console.log(`\n${"=".repeat(50)}`);  
console.log(`共 ${questions.length} 个问题`);
```

神光的编程秘籍

RAG 的 Agent 跑通后，我们来做一些 RAG 的评估。

怎么评估呢？

其实很容易想到：

整理一批问题和对应的标准答案，挨个拿去提问 Agent，对比实际回答和标准回答的差距，以此来完成打分评判。

没错，实际落地也正是这个思路。

我们直接借助 LangSmith 来实现，用里面的 dataset 专门管理评测数据集，统一存放各类问题与标准答案。

再通过 evaluation 配置好各类评估指标与打分规则，就能自动批量发起测试，对照实际输出和标准答案，按照不同维度自动完成评分，高效完成整套 RAG 效果评估。

首先创建 dataset：

src/eval/build_dataset.mjs

```
import "dotenv/config";
import { Client } from "langsmith";

const DATASET_NAME = "rag-eval-v1";

const EXAMPLES = [
  {
    inputs: { question: "无理由退货要在几天内申请?" },
    outputs: { answer: "自签收之日起 7 天内支持无理由退货。" },
  },
  {
    inputs: { question: "质量问题退换货期限是多久?" },
    outputs: { answer: "15 天内出现质量问题可免费换货。" },
  },
  {
    inputs: { question: "无理由退货运费谁承担?" },
    outputs: { answer: "无理由退货由买家承担退货运费。" },
  },
  {
    inputs: { question: "客服工作时间是什么?" },
    outputs: { answer: "周一至周五 9:00-18:00, 周六 10:00-17:00, 法定节假日顺延。" },
  },
  {
    inputs: { question: "满多少元包邮?" },
    outputs: { answer: "满 99 元包邮 (部分大件/冷链除外)。" },
  },
  {
    inputs: { question: "现货商品多久发货?" },
    outputs: { answer: "付款后 24 小时内发货, 大促期间 48 小时内。" },
  },
  {
    inputs: { question: "支持哪些支付方式?" },
    outputs: {
      answer: "支持微信支付、支付宝、银联云闪付、花呗/信用卡分期 (满 500 元可选 3/6/12 期)。",
    },
  },
]
```

```

{
  inputs: { question: "价保是多久? " },
  outputs: { answer: "下单后 7 天内同款降价可申请差价退还。" },
},
{
  inputs: { question: "金卡会员有什么折扣? " },
  outputs: { answer: "金卡享 95 折, 并有专属客服和每月满 200 减 30 券。" },
},
{
  inputs: { question: "积分多少可以抵 1 元? " },
  outputs: { answer: "100 积分可抵 1 元, 单笔最多抵扣实付金额的 30%。" },
},
{
  inputs: { question: "手机保修多久? " },
  outputs: { answer: "手机、平板、耳机全国联保 1 年。" },
},
{
  inputs: { question: "紧急问题怎么联系? " },
  outputs: { answer: "可拨打 400-800-1234 转 2, 接通后报订单号。" },
},
],
];

```

```

asyncfunction main() {
const client = new Client({ apiKey: process.env.LANGCHAIN_API_KEY });

let dataset;
try {
  dataset = await client.readDataset({ datasetName: DATASET_NAME });
  console.log(`数据集已存在: ${DATASET_NAME}`);
} catch {
  dataset = await client.createDataset(DATASET_NAME, {
    description: "RAG Agent 回归评估集",
  });
  console.log(`已创建数据集: ${DATASET_NAME}`);
}

const created = await client.createExamples(
  EXAMPLES.map((e) => ({
    dataset_id: dataset.id,
    inputs: e.inputs,
    outputs: e.outputs,
  })),
);

console.log(`已创建 ${created.length} 条样例`);
}

main().catch((err) => {

```

```
console.error(err);  
  process.exit(1);  
});
```

安装 langsmith 包：

```
pnpm install langsmith
```

跑一下：

神光的编程秘籍

然后我们来创建 evaluator 跑下评估

当然，不用自己创建，langchain 官方提供了 openevals 这个包，内置了很多评估器

安装下：

```
pnpm install openevals
```

创建 src/evals/evaluators.mjs

```
/**  
 * OpenEvals 内置 RAG 指标  
 */
```

```

import {
  createLLMAsJudge,
  RAG_GROUNDEDNESS_PROMPT,
  RAG_HELPFULNESS_PROMPT,
  RAG_RETRIEVAL_RELEVANCE_PROMPT,
} from "openevals";
import { ChatOpenAI } from "@langchain/openai";

const judge = new ChatOpenAI({
  apiKey: process.env.OPENAI_API_KEY,
  configuration: { baseUrl: process.env.OPENAI_BASE_URL },
  model: process.env.MODEL_NAME ?? "qwen-plus",
  temperature: 0,
});

// RAG_GROUNDEDNESS_PROMPT -- 忠实度: 答案是否被检索上下文支撑, 有无幻觉
const ragGroundednessJudge = createLLMAsJudge({
  prompt: RAG_GROUNDEDNESS_PROMPT,
  feedbackKey: "rag_groundedness",
  judge,
  continuous: true,
});

// RAG_HELPFULNESS_PROMPT -- 回答有用性: 是否切题、是否答非所问
const ragHelpfulnessJudge = createLLMAsJudge({
  prompt: RAG_HELPFULNESS_PROMPT,
  feedbackKey: "rag_helpfulness",
  judge,
  continuous: true,
});

// RAG_RETRIEVAL_RELEVANCE_PROMPT -- 检索相关性: 召回片段与问题是否相关
const ragRetrievalRelevanceJudge = createLLMAsJudge({
  prompt: RAG_RETRIEVAL_RELEVANCE_PROMPT,
  feedbackKey: "rag_retrieval_relevance",
  judge,
  continuous: true,
});

export async function ragGroundednessEvaluator({ outputs }) {
  return ragGroundednessJudge({
    context: { documents: outputs.context },
    outputs: { answer: outputs.answer },
  });
}

export async function ragHelpfulnessEvaluator({ inputs, outputs }) {
  return ragHelpfulnessJudge({ inputs, outputs: { answer: outputs.answer } });
}

```



```

}

export async function ragRetrievalRelevanceEvaluator({ inputs, outputs }) {
  return ragRetrievalRelevanceJudge({
    inputs,
    context: { documents: outputs.context },
  });
}

export const ragEvaluators = [
  ragGroundednessEvaluator,
  ragHelpfulnessEvaluator,
  ragRetrievalRelevanceEvaluator,
];

```

我们是用大模型来做评估，openevals 内置了一些评估器可以用，直接引入对应的 prompt 即可，不用自己写。

- RAG_GROUNDEDNESS_PROMPT —— **忠实度**：答案是否被检索上下文支撑，有无幻觉
- RAG_HELPFULNESS_PROMPT —— **回答有用性**：是否切题、是否答非所问
- RAG_RETRIEVAL_RELEVANCE_PROMPT —— **检索相关性**：召回片段与问题是否相关

然后跑一下：

src/evals/run_eval.mjs

```

/**
 * RAG 评测入口: dataset (问题+标准答案) + evaluate
 */

import "dotenv/config";
import { Client } from "langsmith";
import { evaluate } from "langsmith/evaluation";
import { ask } from "../rag_agent.mjs";
import { ragEvaluators } from "../evaluators.mjs";

const DATASET_NAME = "rag-eval-v1";
const client = new Client({ apiKey: process.env.LANGCHAIN_API_KEY });

/** 被评测的 RAG Agent */
async function runRagAgent(inputs) {
  const { answer, context } = await ask(inputs.question);
  return {

```

```

        answer,
        context: context.map((d) => d.pageContent),
    };
}

asyncfunction main() {
const result = await evaluate(runRagAgent, {
    data: DATASET_NAME,
    evaluators: ragEvaluators,
    client,
    experimentPrefix: `rag-openevals-${process.env.MODEL_NAME ?? "qwen"}`,
    maxConcurrency: 2,
});

// 等待全部样例跑完
forawait (const _row of result) {
    /* drain */
}

const project = process.env.LANGCHAIN_PROJECT ?? "default";
console.log("✅ 评测完成");
console.log("实验名:", result.experimentName);
console.log(
    "指标: rag_groundedness | rag_helpfulness | rag_retrieval_relevance",
);
console.log(
    `报告: https://smith.langchain.com/o/default/projects/p/${encodeURIComponent(project)}`
);
}

main().catch((err) => {
    console.error(err);
    process.exit(1);
});

```

跑一下:

这样，我们就能把 RAG 的效果给量化，根据指标来评估。

一目了然看到问题、标准答案、我们的答案，大模型各种维度的打分。

这样跑一次评估叫做一次实验 experiment

RAG 的量化评估，写简历必备的点。

神光的编程秘籍

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

这节我们学了 LangSmith，包括 trace、monitor、dataset、evaluator、experiment 这些概念

trace 就是 Agent 运行过程数据的收集，比如 LangGraph 的 graph，LangChain 的 chain

可以看到每个节点的输入输出，tool 的参数返回值、token 消耗、耗时、报错等数据

只要在环境变量加上 LANGCHAIN_API_KEY、LANGCHAIN_PROJECT 标识用户和项目，然后开启 LANGCHAIN_TRACING_V2 就可以了，不用改代码，自动收集上报数据

monitor 是一些统计指标，比如 tool 调用了几次、token 消耗的变化等

dataset 是问题和标准答案的数据集，可以用它来跑实验

也就是用评估器从各种维度来打分，可以用 langchain 官方提供的 openevals 这个包的内置的 evaluator 评估器

用 langsmith 的包的 evaluate 方法来跑实验，指定 evaluator 和 dataset 就能够对 Agent 的效果做评估

这样有了量化的指标后，就知道我们的 Agent 或者 RAG 效果怎么样了，比如忠实度、回答有用性、检索相关性等指标

Agent 的可观测性以及效果量化评估，是做 Agent 很重要的一个点，也是面试必问的。

转型 Agent 全栈工程师：企业级知识库项目 · 目录

[上一篇](#)

Neo4j 知识图谱和 Graph RAG

[下一篇](#)

DeepAgents：开箱即用的 skill、上下文压缩等 middleware